

# E2E Test Report cho Vibe Coding

Hướng dẫn đầy đủ — kiểm thử sản phẩm AI-generated và viết báo cáo QA ở chuẩn excellence

e2e-test-report

## Lời mở đầu

Bạn vừa dùng AI để build một sản phẩm phần mềm trong vài giờ. Nó “có vẻ hoạt động”. Nhưng “có vẻ” không đủ — đặc biệt khi sản phẩm có AI/chatbot, khi nó phục vụ nhiều vai trò người dùng, khi nó xử lý dữ liệu thật.

Cuốn ebook ngắn này đi cùng **e2e-test-report** — một agent skill biến câu lệnh “test sản phẩm X có hoạt động không” thành một báo cáo QA chuyên nghiệp, có bằng chứng, có điểm số minh bạch. Chúng ta sẽ đi qua: vì sao test sản phẩm vibe coding khác test truyền thống, bộ methodology 5 tầng + 2 trục, cách chấm điểm rubric, và quan trọng nhất — cách tránh những cái bẫy khiến test-runner kết luận sai.

**Triết lý cốt lõi:** “Không tin lời nói — chỉ tin kết quả test — và trình bày bằng chứng ở chuẩn excellence.”

## Chương 1 — Vì sao test vibe coding là một bài toán khác

“Vibe coding” — lập trình bằng cách mô tả ý định cho AI — sinh ra sản phẩm nhanh前所未有. Nhưng tốc độ đó đi kèm 3 đặc điểm khiến kiểm thử khó hơn:

**1. Sản phẩm “trông” xong nhưng chưa chắc hoạt động.** AI giỏi tạo UI trông hoàn chỉnh, route đầy đủ, nút bấm đẹp — nhưng bên trong có thể là placeholder, luồng đứt, hoặc tính năng AI chỉ là ô search đóng vai trợ lý. Mắt thường không phân biệt được.

**2. Nhiều sản phẩm có thành phần AI.** Một chatbot có thể là RAG thật (truy xuất kiến thức, trích dẫn nguồn) — hoặc chỉ là một đoạn greeting canned trả lại giống nhau cho mọi câu. Phân biệt hai loại này cần kỹ thuật riêng.

**3. Tốc độ build khiến spec lung lay.** Tính năng thay đổi nhanh, tài liệu hướng dẫn (HDSD) có thể lệch thực tế. Test theo HDSD trở thành một tầng kiểm thử thiết yếu, không phải optional.

Skill này giải quyết cả 3 bằng cách **bắt buộc bằng chứng** cho mọi nhận định, và dùng một ma trận test nhiều tầng để không bỏ sót.

## Chương 2 — Methodology: ma trận test 5 tầng

Mọi thứ cần test được xếp vào 5 tầng, từ hẹp đến rộng:

- Tầng 1 — Unit:** test từng thành phần nhỏ (1 hàm, 1 endpoint, 1 rule). Input cụ thể → output kỳ vọng.
- Tầng 2 — Integration:** test nhiều thành phần cùng nhau (API + DB, frontend + backend).
- Tầng 3 — Functional:** test tính năng hoàn chỉnh theo specification.
- Tầng 4 — UAT:** test theo kịch bản sử dụng thực tế của người dùng.
- Tầng 5 — Documentation Compliance:** đọc HDSD → làm theo → xác minh kết quả đúng như mô tả.

### Quy tắc fail-fast

Khi chạy full suite, test theo thứ tự từ thấp đến cao. Nếu tầng thấp fail >50% → **dừng**, báo user, không test tầng cao hơn. Lý do: lỗi tầng thấp sẽ gây lỗi tầng cao, test thêm là vô ích và làm nhiều báo cáo.

### Khi nào dùng tầng nào?

| Câu lệnh của user               | Tầng                     |
|---------------------------------|--------------------------|
| “Test hàm/component này”        | Unit                     |
| “Test kết nối A↔B”              | Integration              |
| “Test tính năng X đúng spec”    | Functional               |
| “Test theo kịch bản người dùng” | UAT                      |
| “Test theo HDSD”                | Documentation Compliance |
| “Test toàn diện” / không chỉ rõ | Tất cả 5 tầng            |

## Chương 3 — Software Product QA Report: 2 trục song song

Khi target là một **sản phẩm phần mềm hoàn chỉnh** (web app đa role, SaaS, chatbot), skill dùng schema báo cáo chuyên nghiệp với 2 trục song song:

**Trục A — QA toàn hệ thống (functional).** Login đủ role, click toàn luồng, soi Network & Console, test responsive. Output: điểm /100 + bộ đếm severity + bug cards chi tiết.

**Trục B — Độ chính xác AI/RAG (chỉ khi có chatbot).** Chạy hội thoại đa lượt, đối chiếu từng câu trả lời với nguồn chính thức (ground truth) để xếp loại Đúng / Một phần / Sai. Output: điểm /10 + bảng chấm accuracy + findings.

*Nếu sản phẩm không có chatbot → Trục B không áp dụng, trọng số C4 (AI Quality) dời sang C3 và C5.*

### Bug card — 5 trường bắt buộc

Mỗi lỗi hệ thống là một card với: **Hiện tượng** · **Kịch bản tái hiện** · **Nguyên nhân gốc** · **Ghi chú dev** · **Ảnh chứng cứ**. Đây là phần cốt lõi — phải tái hiện được, chỉ ra root cause thật (không phải triệu chứng), và gợi ý fix theo thứ tự ưu tiên.

## Chương 4 — Feature Explore: không test hời hợt

Đây là bài học đắt giá: test mà không khám phá hết feature = chấm hời hợt. Báo cáo chỉ test homepage + login rồi kết luận “sản phẩm OK” là thất bại.

Skill bắt buộc một phase **Feature Explore** trước khi test:

- Crawl nav/routes theo từng role.** Mỗi role thấy nav khác nhau (RBAC surfaces feature khác nhau) → crawl từng role riêng, dùng fresh browser context mỗi role.
- Enumerate mỗi route** → ghi tên feature, route, role, loại (CRUD/chat/dashboard/form/...), screenshot.
- Lập Feature Inventory** — bảng liệt kê mọi feature phát hiện được.

4. **Test từng feature** trong inventory → ghi PASS/FAIL/UNVERIFIED + screenshot + map về tiêu chí rubric.

***KHÔNG** kết luận “product OK” khi còn feature UNTESTED trong inventory. Feature gated/không test được → ghi UNVERIFIED, không bỏ qua.*

## Chương 5 — Rubric chấm điểm /100

Điểm số một mình không đủ — phải có evidence. Skill dùng rubric 7 tiêu chí có trọng số:

| #  | Tiêu chí                           | Trọng số |
|----|------------------------------------|----------|
| C1 | Feature Discovery & Coverage       | 15%      |
| C2 | Auth & Access Control              | 15%      |
| C3 | Core Functionality                 | 25%      |
| C4 | AI Quality (nếu có chatbot)        | 15%      |
| C5 | Reliability & Production-readiness | 15%      |
| C6 | UX & Polish                        | 10%      |
| C7 | Deploy & Testability               | 5%       |

Mỗi tiêu chí chấm **level 1-5** (0 = UNVERIFIED, loại khỏi aggregate). Mức định tính có descriptor khách quan — không chấm theo cảm tính.

### Công thức aggregate (tính lại bằng code)

```
sub_score_i = (level_i / 5) × 100
crit_score = Σ(sub_score × trọng số con) / Σ(trọng số con)
TOTAL = Σ(crit_score × trọng số chính)
```

Tổng điểm **phải tính lại bằng script**, không tin số LLM tự nhảm — đây là một quy tắc anti-hallucination quan trọng.

### Độ phủ + confidence trung thực

Điểm “đẹp” có thể che việc chỉ test được 45% tiêu chí. Skill hiển thị **độ phủ** (% tiêu chí kiểm tra được) và **confidence** (0.55-0.92 tùy tỷ lệ UNVERIFIED). Confidence thấp (<0.6) khi >50%

UNVERIFIED → ghi rõ, không present điểm như verified.

# Chương 6 — Proof Layer: “không test được” ≠ “bị hỏng”

Đây là chương quan trọng nhất. Lỗi nguy hiểm nhất của test tự động là **kết luận “phần mềm lỗi” khi thực ra là test-runner bất lực hoặc đo sai.**

## 5 nguyên tắc Proof

- 1. **Đa phương pháp độc lập (triangulation)** — mọi kết luận quan trọng (login, AI, core) phải có ≥2 phương pháp đo độc lập. Một heuristic keyword là không đủ.
- 2. **PASS cần proof dương, FAIL cần proof âm về phần mềm.** PASS = bằng chứng tích cực (đích reached, output match). FAIL = chứng minh phần mềm gây lỗi, sau khi test-runner đã tới đúng trạng thái.
- 3. **“Không test được” ≠ “Bị hỏng”.** Gated / không account / timeout / SPA chưa render → ghi **UNVERIFIED**, KHÔNG BAO GIỜ ghi FAIL.
- 4. **Self-check test-runner.** Trước khi kết luận feature X hỏng, chứng minh harness đã tới đúng trạng thái (login verified bằng token/role-content; AI ở đúng route không nhầm search).
- 5. **Deterministic > heuristic.** Ưu tiên HTTP status / element existence / JSON field / role-nav-diff hơn keyword-matching body text.

## Bảng: đừng kết luận X, hãy kết luận Y

| Tình huống                           | ❌ Sai              | ✅ Đúng                                               |
|--------------------------------------|--------------------|------------------------------------------------------|
| Không có account                     | “Login hỏng”       | “UNVERIFIED — thiếu account”                         |
| SPA rỗng sau domcontentloaded        | “White-screen bug” | Verify lại với networkidle; vẫn rỗng → FAIL có proof |
| Cookie trống (app dùng localStorage) | “Chưa login”       | Check localStorage + role-specific content           |
| URL không đổi sau login (SPA)        | “Login fail”       | Check token store + role nav diff                    |

# Chương 7 — Harness playbook: kỹ thuật test web-app thật

---

Proof Layer là nguyên tắc; phần này là kỹ thuật cụ thể. Mỗi kỹ thuật sinh ra từ một lỗi thật:

**Kỹ thuật 1 — Fresh context per role.** Test nhiều role trong cùng 1 browser context → session role đầu còn persistent → role sau “fail” giả. Sửa: mỗi role một context mới.

**Kỹ thuật 2 — Login verify 3-method.** URL không đổi ≠ login fail (SPA giữ URL). Cookie trống ≠ chưa login (app dùng localStorage). Đúng: verify bằng token store + role-nav diff + screenshot.

**Kỹ thuật 3 — AI element detection.** Chỉ tìm `textarea` → bỏ lỡ `input[type=text]`. Lại test nhầm ô `/search` → kết luận “AI = search”. Đúng: tìm input có placeholder chat, loại placeholder “tìm”/“search”.

**Kỹ thuật 4 — Capture bot bubble.** Lấy `body[-500:]` làm “phản hồi bot” → lấy text footer/dashboard → chấm AI sai. Đúng: tìm message bubble thật (25-1500 ký tự, không chứa câu gốc/footer).

**Kỹ thuật 5 — ≥2 câu hỏi khác nhau.** Hỏi “Xin chào” → bot trả greeting → tưởng “AI tốt”. Đúng: gửi 1 câu domain + 1 câu out-of-scope; nếu cùng text → canned/scripted.

**Kỹ thuật 6 — Backend health curl độc lập.** Frontend 200 nhưng báo “backend chết” → sai. Đúng: curl trực tiếp `/api/health`, tách rời frontend render.

**Kỹ thuật 9.5 — On-page credential discovery.** Spec không có account → vội ghi “thiếu account”. Nhưng nhiều sản phẩm để credentials ngay trang login (panel demo, hint) hoặc có button Demo/Showcase. Quét 4 nguồn (DOM text, demo button, form prefilled, route `/demo`) TRƯỚC khi ghi UNVERIFIED.

---

# Chương 8 — Report format excellence

---

Báo cáo phải đẹp, không chỉ đúng. Report docx từ pandoc mặc định rất khó xem (font nhạt, bảng thô, ảnh quá to). Skill dùng **python-docx** để kiểm soát đầy đủ:

- **Cover page** — bảng màu navy + tên sản phẩm lớn + bảng thông tin + **score badge nổi bật** (số điểm lớn trong ô màu, kèm độ phủ + band). Nhìn 1 cái là biết điểm.
- **Color-coded score** — xanh ≥70, vàng 50-69, đỏ <50. Áp cho badge, cell rubric, band.
- **Rubric table styled** — header navy, cell “cấp” tô màu, hàng total highlight.

- **Card test chi tiết (BẮT BUỘC)** — mỗi tiêu chí C1-C7 = 1 card: header màu theo verdict + bảng Kịch bản/Expected/Actual + ảnh proof. Đây là phần thuyết phục nhất.
- **Screenshot sized + captioned** — width ~4.6-6 inch, caption italic xám.
- **Page footer** — tên sản phẩm + số trang.

Builder tham chiếu: `report-template/build_report_docx.py`. Chạy `python3 build_report_docx.py` là ra report.

## Chương 9 — Cài đặt và Quickstart

### Cài đặt (ưu tiên Claude Code)

```
# Claude Code
unzip e2e-test-report.zip -d ~/.claude/skills/

# Codex CLI: copy nội dung SKILL.md vào ~/.codex/AGENTS.md

# Antigravity: đặt thư mục vào ~/.antigravity/skills/ (hoặc workspace .skills/)
```

Packages (cho harness + report builder):

```
pip3 install --break-system-packages python-docx playwright
python3 -m playwright install chromium
```

### Pipeline 6 bước

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| 0. SETUP           | install + cấu hình qa_teams.py (URL + account các role) |
| 1. qa_harness      | login verify + AI route detection + screenshot          |
| 2. feature_explore | crawl nav → Feature Inventory                           |
| 3. ai_probe        | (có chatbot) accuracy + anti-hallucination              |
| 4. backend health  | curl /api/health đọc lập frontend                       |
| 5. rubric scoring  | gán level 1-5 từ evidence, aggregate recompute          |
| 6. build_report    | docx đẹp: cover + badge + rubric + card chi tiết + ảnh  |

Hoặc đơn giản nhất: gọi `/e2e-test-report <URL>` trong client AI và để skill dẫn bạn.

## Chương 10 — Troubleshooting & FAQ

**Q: Skill báo “login fail” nhưng tôi chắc account đúng.** A: Có thể skill dùng heuristic sai. SPA giữ URL sau login → URL không đổi không phải fail. App dùng localStorage → cookie trống không phải chưa login. Skill verify bằng token store + role-nav diff để tránh cái bẫy này. Nếu vẫn fail → check backend auth thật (curl endpoint).

**Q: Sản phẩm gated, không có account → báo FAIL?** A: Không. Skill ghi **UNVERIFIED** (đã quét on-page demo/showcase mode trước). “Không test được” ≠ “bị hỏng”. Cung cấp account hoặc mở demo mode để re-verify.

**Q: Skill chấm AI điểm thấp dù chatbot trả lời lịch sự.** A: Có thể là greeting canned — cùng text cho mọi câu. Skill gửi ≥2 câu khác nhau (domain + out-of-scope); nếu cùng phản hồi → scripted, không generative. Out-of-scope mà bot bịa → hallucination.

**Q: Điểm cao nhưng độ phủ thấp.** A: Điểm chỉ phản ánh phần đã kiểm tra. Nhìn cột **độ phủ** + **confidence**. Phủ thấp → cần account test để re-verify các tiêu chí UNVERIFIED.

**Q: Screenshot bị lẫn cửa sổ khác.** A: Skill dùng window-specific capture ( `screenshot -l <CGWindowID>` ), KHÔNG fullscreen — tránh lộ PII/sensitive info từ app đang chạy khác.

---

## Kết

---

`e2e-test-report` không phải công cụ “chạy rồi nói có vẻ ổn”. Nó là một Test Architect: thiết kế ma trận test, tạo môi trường độc lập, chạy từng kịch bản, ghi nhận chính xác pass/fail với bằng chứng, và dọn dẹp sạch sau khi xong. Báo cáo nó sinh ra — đẹp, có điểm số minh bạch, có ảnh proof — đủ thuyết phục cả người kỹ thuật lẫn người không kỹ thuật.

Bắt đầu tại: `e2e-test-report/SKILL.md` . Mở `samples/QA-Report-Sample.docx` để thấy một report Excellence trông thế nào.

*“Không tin lời nói — chỉ tin kết quả test.”*

---

MIT License — dùng tự do. Sản phẩm mẫu DemoKB là hư cấu, không liên quan sản phẩm thật.